

American International University-Bangladesh

Faculty of Science and Technology
Department of Computer Science

CSC 3114 Software Engineering

Scenario-Based Questions on Software Development Process Models

Mid Term Examination Spring 2025-26

Total Marks: 60

Faculty: Mejbah Ahammad

Time: 2 Hours

Waterfall Model

Scenario 1: A government organization plans to develop a national tax management system where all requirements are clearly defined after extensive stakeholder discussions. The project follows a strict sequence of phases including requirement analysis, design, implementation, testing, and deployment, and changes are not expected once development begins.

Questions:

1. Identify the most appropriate software process model.
2. Explain why this model is suitable for this situation.
3. Discuss the advantages of using this model.
4. Analyze the risks if requirements change during later stages.
5. Suggest whether any improvement or alternative approach should be considered.

Scenario 2: A bank is developing a financial accounting system where all regulatory requirements are fixed and must be strictly followed. The development team proceeds step by step, completing one phase before moving to the next, with heavy emphasis on documentation and approvals.

Questions:

1. Identify the most appropriate software process model.
2. Justify why this model is suitable for a regulated environment.
3. Discuss how documentation benefits this model.
4. Explain the limitations if customer needs evolve.
5. Propose an alternative if flexibility becomes necessary.

Scenario 3: A software company starts building a system assuming that requirements are fixed. However, after the design phase, the client requests major changes in functionality, causing delays and rework.

Questions:

1. Identify the model being used.
2. Explain why this model struggles with changing requirements.
3. Analyze the impact of late changes on cost and time.
4. Discuss the risks associated with this model in dynamic environments.
5. Suggest a more suitable model for such situations.

Scenario 4: A defense organization is developing a secure communication system where each phase must be approved before proceeding to the next. The process is strictly controlled and sequential, with no overlap between phases.

Questions:

1. Identify the development model used.
2. Explain why a sequential approach is required in this scenario.
3. Discuss the advantages of strict phase control.
4. Evaluate the limitations of this approach.
5. Suggest how risk can be minimized in this model.

Scenario 5: A small software project with clearly defined and stable requirements is developed by a team following a linear sequence of steps from requirement gathering to deployment without revisiting earlier stages.

Questions:

1. Identify the process model used.
2. Explain why this model is suitable for small and stable projects.
3. Discuss the benefits of simplicity in this model.
4. Analyze potential drawbacks if assumptions fail.
5. Suggest improvements for better adaptability.

V-Model

Scenario 1: A medical software system is being developed where each development phase must have a corresponding testing phase. The system must be highly reliable because it will be used for patient diagnosis and treatment decisions.

Questions:

1. Identify the most appropriate software process model.
2. Explain how this model ensures verification and validation.
3. Describe the relationship between development and testing phases.
4. Discuss why this model is suitable for safety-critical systems.

5. Analyze its limitations in handling requirement changes.

Scenario 2: A project team follows a structured approach where requirements are directly linked to acceptance testing, system design to system testing, and module design to unit testing. Each stage is validated before moving forward.

Questions:

1. Identify the process model used.
2. Explain how testing is integrated into each phase.
3. Discuss the advantages of early test planning.
4. Evaluate how this model improves software quality.
5. Identify potential drawbacks of this approach.

Scenario 3: A defense system requires strict validation at every stage of development. The project ensures that testing activities are planned alongside development activities to avoid errors in later stages.

Questions:

1. Identify the most suitable model.
2. Explain how this model supports early defect detection.
3. Discuss the importance of validation in this context.
4. Analyze why this model is preferred over Waterfall.
5. Suggest limitations in terms of flexibility.

Scenario 4: A project follows a sequential development process with corresponding testing phases, but during development, frequent changes in requirements are introduced, causing difficulties in maintaining alignment between development and testing.

Questions:

1. Identify the model being used.
2. Explain why this model struggles with changing requirements.
3. Analyze the impact of requirement changes on testing phases.
4. Discuss risks associated with this situation.
5. Suggest a more flexible alternative model.

Scenario 5: A large enterprise system is developed where strict quality assurance policies require that each phase be verified and validated before proceeding to the next phase. The organization emphasizes defect prevention rather than detection after development.

Questions:

1. Identify the development model used.

2. Explain how this model emphasizes quality assurance.
3. Discuss the advantages of verification and validation.
4. Evaluate the suitability of this model for large systems.
5. Suggest possible improvements for handling dynamic requirements.

Prototyping Model

Scenario 1: A startup is developing a mobile application, but users are unsure about their exact requirements. The development team creates an initial prototype and presents it to users, collects feedback, and refines the system repeatedly until the requirements become clear.

Questions:

1. Identify the most appropriate software process model.
2. Explain why this model is suitable for unclear requirements.
3. Describe the role of user feedback in this model.
4. Discuss how iteration improves the final system.
5. Analyze potential risks associated with this approach.

Scenario 2: A company builds a quick working model of a new system to demonstrate functionality to clients before actual development begins. The prototype is modified several times based on client suggestions.

Questions:

1. Identify the process model used.
2. Explain how this model helps in requirement gathering.
3. Discuss the advantages of early visualization.
4. Evaluate how client involvement impacts development.
5. Identify possible limitations of this model.

Scenario 3: A prototype developed for a system is directly converted into the final product without proper redesign or optimization, leading to performance and scalability issues.

Questions:

1. Identify the model being used.
2. Explain why directly using a prototype as the final system can be problematic.
3. Analyze the risks in terms of system quality.
4. Discuss how proper design practices can mitigate these issues.

5. Suggest improvements to avoid such problems.

Scenario 4: A software project involves continuous changes in requirements as users better understand their needs after interacting with intermediate versions of the system.

Questions:

1. Identify the most suitable development model.
2. Explain how this model supports evolving requirements.
3. Discuss the importance of iterative refinement.
4. Analyze how this approach reduces requirement uncertainty.
5. Evaluate its impact on project timeline and cost.

Scenario 5: A design team develops multiple versions of a user interface prototype to test usability and user experience before finalizing the system design.

Questions:

1. Identify the development model used.
2. Explain how prototyping improves usability.
3. Discuss the benefits of testing design concepts early.
4. Analyze the role of user interaction in refining the system.
5. Suggest limitations of relying heavily on prototypes.

Incremental Model

Scenario 1: A company develops an e-commerce platform in stages. The first release includes user registration and login, the second includes product browsing, the third includes payment processing, and later versions add recommendation features. Each version is delivered as a working product.

Questions:

1. Identify the most appropriate software process model.
2. Explain why this model is suitable for this project.
3. Discuss how early delivery benefits users.
4. Analyze how this model manages complexity.
5. Evaluate potential integration challenges.

Scenario 2: A software system is divided into smaller modules, and each module is developed, tested, and delivered separately. The core features are developed first, while additional features are planned for later releases.

Questions:

1. Identify the process model used.

2. Explain how prioritization of features works in this model.
3. Discuss advantages of delivering core functionality early.
4. Analyze how this model handles changing requirements.
5. Identify possible drawbacks of modular development.

Scenario 3: A project begins with a well-defined set of core requirements, but additional features are expected to evolve over time. Once development of an increment starts, its requirements are frozen, while future increments remain flexible.

Questions:

1. Identify the development model used.
2. Explain the concept of freezing requirements in increments.
3. Discuss how flexibility is maintained for future increments.
4. Analyze the impact on project planning and scheduling.
5. Suggest how risks can be minimized in this approach.

Scenario 4: A large system is developed over multiple releases, but integration between different increments causes compatibility issues and delays.

Questions:

1. Identify the model being used.
2. Explain why integration challenges occur in this model.
3. Discuss strategies to manage integration effectively.
4. Analyze the risks of poor coordination between increments.
5. Suggest improvements to ensure smooth integration.

Scenario 5: A company wants to deliver a system quickly to gain user feedback while continuing development of additional features. Each release improves the system based on user input.

Questions:

1. Identify the most suitable model.
2. Explain how this model supports continuous improvement.
3. Discuss the role of user feedback in later increments.
4. Analyze the benefits of iterative delivery.
5. Evaluate limitations in terms of long-term planning.

Rapid Application Development (RAD) Model

Scenario 1: A software company is required to develop a university management system within a very short time frame of three months. The system is divided into modules such as student management, course registration, and billing, and multiple teams work on these modules simultaneously.

Questions:

1. Identify the most appropriate software process model.
2. Explain why this model is suitable for time-constrained projects.
3. Discuss the role of parallel development in this model.
4. Analyze how this approach accelerates delivery.
5. Evaluate potential risks associated with rapid development.

Scenario 2: A project is developed using time-boxed phases where each component is built quickly and integrated into a working system. Users are continuously involved to provide feedback during development.

Questions:

1. Identify the process model used.
2. Explain the concept of time-boxing in this model.
3. Discuss the importance of user involvement.
4. Analyze how this model improves responsiveness to feedback.
5. Identify limitations of this approach.

Scenario 3: Multiple teams work independently on different modules of a system, treating each module as a mini-project. These modules are later integrated into a complete system within a short time frame.

Questions:

1. Identify the development model used.
2. Explain how modular and parallel development works in this model.
3. Discuss advantages of dividing work into mini-projects.
4. Analyze integration challenges that may arise.
5. Suggest strategies to manage coordination between teams.

Scenario 4: A project developed under strict deadlines results in poor system design and maintainability due to rushed development and insufficient planning.

Questions:

1. Identify the model being used.
2. Explain why this model can lead to poor design under pressure.

3. Analyze risks associated with insufficient planning.
4. Discuss how quality can be compromised in rapid development.
5. Suggest improvements to balance speed and quality.

Scenario 5: A company with highly skilled developers and strong tool support develops a system quickly by reusing components and focusing on fast delivery while involving users throughout the process.

Questions:

1. Identify the most suitable model.
2. Explain how skilled teams contribute to success in this model.
3. Discuss the role of reusable components.
4. Analyze the benefits of continuous user interaction.
5. Evaluate limitations when resources or expertise are lacking.

Rational Unified Process (RUP) Model

Scenario 1: A large banking system is being developed with high complexity and risk. The project is divided into phases: inception, elaboration, construction, and transition. The team continuously evaluates risks and refines the system through multiple iterations.

Questions:

1. Identify the most appropriate software process model.
2. Explain why this model is suitable for complex and high-risk systems.
3. Describe the purpose of each phase in this model.
4. Analyze how iterative development improves system quality.
5. Discuss the role of risk management in this process.

Scenario 2: A project is developed in multiple iterations, where each iteration produces a working version of the system. Risks are identified early and addressed throughout the development lifecycle.

Questions:

1. Identify the process model used.
2. Explain how iteration is structured in this model.
3. Discuss the benefits of addressing risks early.
4. Analyze how continuous refinement improves the system.
5. Evaluate limitations of this approach.

Scenario 3: A company develops a large enterprise system where initial planning focuses on feasibility and scope, followed by detailed architecture design and risk analysis before full-scale development begins.

Questions:

1. Identify the development model used.
2. Explain the importance of the inception and elaboration phases.
3. Discuss how architecture planning reduces future risks.
4. Analyze how this model differs from linear models.
5. Suggest advantages of early risk identification.

Scenario 4: A system evolves through repeated cycles of development, testing, and feedback, allowing gradual improvement while managing uncertainties and risks.

Questions:

1. Identify the model being used.
2. Explain how iterative cycles benefit development.
3. Discuss how this model handles uncertainty.
4. Analyze the importance of continuous feedback.
5. Evaluate challenges in managing multiple iterations.

Scenario 5: A project requires flexibility, continuous improvement, and strong risk management, but also needs structured phases and clear milestones for progress tracking.

Questions:

1. Identify the most suitable model.
2. Explain how this model balances structure and flexibility.
3. Discuss the importance of defined phases.
4. Analyze how risk management is integrated.
5. Compare this model with the Waterfall model in terms of adaptability.

Solutions to Scenario-Based Questions

Waterfall Model Solutions

Scenario 1: The most appropriate model for this scenario is the Waterfall Model. This model is suitable because the project requirements are clearly defined and stable from the beginning. In such environments, a linear and structured approach ensures that each phase is completed thoroughly before moving to the next. The emphasis on documentation and formal approvals aligns well with government projects. One of the major advantages is predictability in scheduling and cost estimation. However, a significant

limitation arises if requirements change later, as the model does not support backtracking easily. Any modification would require revisiting earlier phases, increasing both time and cost.

Scenario 2: The Waterfall Model is ideal for this regulated banking system due to its strict adherence to predefined requirements and documentation. Regulatory environments demand traceability, and Waterfall ensures that each phase is documented and validated. This helps in compliance audits and ensures accountability. However, the rigidity of the model means that adapting to new financial regulations or customer needs can be challenging. Therefore, while the model ensures stability, it lacks flexibility.

Scenario 3: The project described follows the Waterfall Model, which becomes unsuitable when requirements change after development has begun. Since each phase depends on the completion of the previous one, late changes disrupt the entire process. This leads to increased development costs, delays, and potential inconsistencies in the system. A more flexible model such as Prototyping or Incremental would be better suited for such environments.

Scenario 4: The Waterfall Model is used in this defense system due to its strict sequential nature and emphasis on approvals. Each phase must be validated before proceeding, ensuring high security and reliability. This structured approach reduces risks in critical systems. However, the lack of flexibility can be a drawback if unexpected changes occur. Risk can be minimized by conducting thorough requirement analysis before development begins.

Scenario 5: The Waterfall Model is suitable for small projects with well-defined requirements. Its simplicity makes it easy to manage and implement. The clear structure helps teams stay organized and focused. However, if initial assumptions about requirements are incorrect, the model becomes inefficient due to its inability to adapt.

V-Model Solutions

Scenario 1: The V-Model is the most appropriate choice for this medical system because it integrates testing into every stage of development. Each development phase has a corresponding testing phase, ensuring that defects are identified early. This is crucial in safety-critical systems where failure can have severe consequences. The model emphasizes both verification and validation, improving reliability.

Scenario 2: This scenario clearly represents the V-Model, where development phases are directly linked to testing phases. This approach ensures that testing is planned early and executed systematically. Early defect detection reduces cost and improves software quality. However, the model is less flexible when requirements change.

Scenario 3: The V-Model supports early defect detection through its structured testing approach. Validation ensures that the system meets user needs, while verification ensures correctness of implementation. Compared to Waterfall, it provides better quality assurance but still lacks flexibility.

Scenario 4: The V-Model struggles in environments with frequent requirement changes because its tightly coupled phases make adjustments difficult. Changes in requirements require updates in both development and corresponding testing phases, leading to delays and increased complexity.

Scenario 5: The V-Model emphasizes defect prevention through early testing and validation. It is suitable for large systems requiring high quality. However, its rigid structure makes it less suitable for dynamic environments.

Prototyping Model Solutions

Scenario 1: The Prototyping Model is ideal when requirements are unclear. By creating an initial version of the system, users can interact with it and provide feedback. This iterative process helps refine requirements and ensures that the final system meets user expectations.

Scenario 2: This model allows developers to visualize system functionality early. Client involvement ensures that the system aligns with user needs. This reduces misunderstandings and improves requirement accuracy.

Scenario 3: Using a prototype as the final system can lead to poor design and performance issues because prototypes are usually built quickly without considering scalability or maintainability. Proper redesign and optimization are necessary before final implementation.

Scenario 4: The Prototyping Model supports evolving requirements through continuous refinement. Each iteration improves the system based on user feedback, reducing uncertainty and improving user satisfaction.

Scenario 5: Prototyping improves usability by allowing designers to test different interface designs. User interaction helps identify usability issues early. However, excessive reliance on prototypes can lead to increased development time and cost.

Incremental Model Solutions

Scenario 1: The Incremental Model is suitable for this e-commerce system as it allows delivery of functional components in stages. Early releases provide value to users and allow feedback for future improvements.

Scenario 2: This model prioritizes core features, ensuring that essential functionality is delivered first. It allows flexibility in developing additional features later.

Scenario 3: Freezing requirements for each increment ensures stability, while allowing future increments to evolve provides flexibility. This balance helps manage complexity effectively.

Scenario 4: Integration challenges arise when independently developed modules do not align properly. Proper planning, interface design, and testing are required to mitigate these issues.

Scenario 5: The model supports continuous improvement by incorporating user feedback into later increments. However, long-term planning can be complex due to evolving requirements.

RAD Model Solutions

Scenario 1: The RAD Model is suitable for time-constrained projects as it emphasizes rapid development and delivery. Parallel development of modules reduces overall development time.

Scenario 2: Time-boxing ensures that each phase is completed within a fixed timeframe, promoting efficiency. Continuous user involvement ensures that the system meets user needs.

Scenario 3: Modular development allows teams to work independently, improving speed. However, integration requires careful coordination.

Scenario 4: Rushed development can lead to poor system design and maintainability issues. Lack of proper planning increases risk.

Scenario 5: RAD works best with skilled teams and reusable components. It enables fast delivery but depends heavily on expertise and resources.

RUP Model Solutions

Scenario 1: The RUP Model is ideal for complex systems due to its iterative and risk-driven approach. Each phase focuses on reducing risk and improving system quality.

Scenario 2: Iteration allows continuous refinement and delivery of working versions. Early risk identification improves project success.

Scenario 3: Inception and elaboration phases ensure proper planning and architecture design. This reduces future development risks.

Scenario 4: Iterative cycles allow gradual improvement and adaptation to changes. Continuous feedback enhances system quality.

Scenario 5: RUP balances structure and flexibility. Unlike Waterfall, it supports iteration and adaptability, making it suitable for dynamic environments.